Problems I expect reviewers to have: MDP is a tiny fraction of core energy (0.003arguably only impactful gains are with no MDP. but if its so small, why not just have one? - possible way around this is to center results on the A14 first then refer to the no mdp results as almost like a bonus

Guidelines for Submission to MICRO 2026

MICRO 2026 Submission #**NaN** – Confidential Draft – Do NOT Distribute!!

Abstract

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

1 Introduction

Modern processors increasingly deploy heterogeneous and efficiency-oriented core designs. Contemporary systems commonly combine large high-performance cores with smaller, energy-efficient out-of-order cores, as seen in ARM big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. Efficiency-focused cores are also widely used in power-constrained systems including smartphones, tablets, smartwatches and other edge systems. These platforms prioritise energy efficiency and area usage while still enabling aggressive speculation and out-of-order (OoO) execution. Despite these constraints, many speculation mechanisms in these cores operate unconditionally on every relevant instruction. To maximise the energy-efficiency of speculative techniques, predictors should ideally only be invoked for instructions that are known to benefit.

Load instructions exemplify this problem. Prior work demonstrates that a considerable portion of load instructions in general-purpose workloads rarely or never exhibit in-flight memory dependencies [**? ? ?**]. Memory dependence prediction (MDP) is a speculative technique used by CPUs to stall loads that have memory dependencies on the appropriate store, while allowing memory-independent loads to schedule as soon as possible [**? ?**]. Despite the ability of most loads to always issue immediately, they all query the MDP on every execution. This is not only inefficient but can also lead to false dependencies due to hash collisions, limiting performance.

We demonstrate that this observation can be exploited to greatly enhance the energy efficiency of the Memory Dependence Predictor (MDP). We implement a profile-guided optimisation technique that labels frequently memory-independent loads by their opcode and simulate the modified binaries using Gem5. These labelled load instructions bypass MDP queries and are always scheduled as early as possible. We show that preventing unnecessary predictions greatly reduces predictor activity while maintaining or even improving IPC.

1.1 Contributions

This paper makes the following contributions:

- Implements a PGO technique to label memory independent loads to the OoO core at no-cost communication or hardware overhead.
- Implements a modified Gem5 to simulate labelled loads against different MDP algorithms and CPU size configurations, as well as extends McPat to provide power usage estimations of the MDP unit.
- Demonstrates that across Spec2017 intspeed:
- An average of 72% (up to 96%) of MDP load queries are either redundant or detrimental to performance,
- Preventing these queries can yield an IPC gain between 0.47% and 3.2% on smaller cores, while maintaining performance on larger cores,
- Average MDP power usage can be reduced between 7% and 42% depending on core size.

2 Background & Related Work

- LSQ, MDP, store sets, PHAST
- my original ARCS paper
- store distances paper & comparison to us
- constable
- LLVM store queue search skip paper
- LSQ scalability labelling paper

3 Method

- justifying PGO: how PGO is commonly used and static analysis (even SVF) doesn't work

4 Evaluation

- experimental design
– cpu size selection reasoning
- IPC
- Lookups
- Energy
- False deps reduction? - PGO & threshold distance scaling
- Prior work comparison

This section presents the experimental results using our PGO technique. It highlights the extent of redundant memory dependence predictor queries found in general-purpose workloads and their impact on IPC and power usage. Our results demonstrate that a large portion of predictor activity can be avoided, leading to improvements in performance, power usage, and even that PND load labels can serve as a partial replacement for the memory dependence predictor unit, all with zero hardware overhead. Furthermore, we examine the generalisability of our generated store distance profiles to unseen inputs and provide a comparison to similar prior work [**?**].

4.1 Experimental Design

We use Gem5 to simulate various CPU size configurations, using parameters from real-world CPUs. We examine the impact of PND

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX

load labels across small to large CPUs, as well as their interaction with different memory dependence predictor algorithms. Three MDP algorithms are employed: no speculation for the most constrained case, Store Sets for smaller cores, and PHAST for larger cores. While PHAST demonstrates greater accuracy, it is less suited to area-constrained cores due to several factors: it requires greater area overhead to perform well, consumes more power for the same area, and incurs increased pipeline complexity to provide predictions. Additionally, smaller cores have limited potential performance gains from enhanced memory dependence prediction. Thus, we consider Store Sets a reasonable choice for the A14 and A725, and use PHAST for the X925 and M4.

Table 1: Parameters of processor components for each simulated model

		S5	A14	M4
Instruction Window	**ROB:**	90	198	986
	IQ:	60	90	456
	LQ:	30	42	128
	SQ:	15	18	72
Pipeline Width	**Fetch:**	90	198	986
	Decode:	60	90	456
	Issue:	30	42	128
	Commit:	15	18	72
Cache	**L1D:**	90	198	986
	L1I:	60	90	456
	L2:	30	42	128
Store Sets	**SSIT:**	N/A	198	N/A
	LFST:		90	
	Clear Period:		42	
PHAST	**Rows**	N/A	198	192
	Assoc:		90	90
	Tag Bits:		42	42
	Counter Bits:			

The full configuration for each model is detailed in Table 1. All CPUs except the S5 utilise a 64KB TAGE-SC-L for branch prediction. To more accurately model the constrained size of the S5, it employs a tournament predictor with 2k local and 8k global entries. We evaluate the Spec2017 intspeed benchmarks using up to 10 simpoints, with an interval of 100M instructions and a warm-up of 10M. Gem5 is modified to bypass MDP lookups for labelled loads and to avoid creating new entries in the case of a memory order violation. Violating labelled loads still roll back as normal and do not alter program semantics. The Gem5 results are subsequently processed by an extended McPat, which models Store Sets or PHAST as applicable (or upstream McPat for the S5).

4.2 IPC

4.3 MDP Queries

4.4 Power

5 Threats to Validity

- intended for specific target compilation

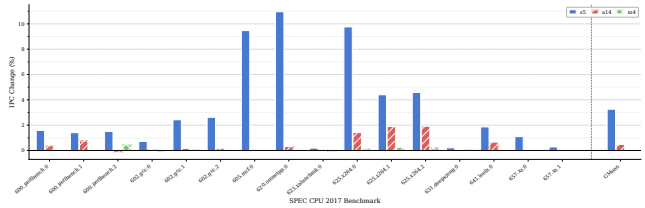

Figure 1: IPC % changes for each CPU model on each intspeed benchmark.

6 Future Work

-serverless workloads
-JITs

7 Conclusion